

VC Generation

References And Borrowing

Goal

Extend Tiny TAC with references without adding a new VCGen core.

Plan:

- model a reference as an explicit value
- lower borrow commands to ordinary assignments, map reads, map writes, and assumes
- run the existing Tiny TAC VCGen

The Puzzle This Deck Solves

```
entry:
  M := havoc
  i := havoc
  r, M2 := borrow_mut M[i]
  r2 := put_ref r, 7
  release r2
  x := M2[i]
  ok := x == 7
  assert ok
  halt
```

`borrow_mut` returns the reference **and** the continuation memory `M2` — in SSA, before any write through `r` has happened.

`x` is read from `M2`, which was created **before** the 7 was written. Why should `assert ok` hold?

The answer is the trick of this deck.

Reference Types

Tiny TAC gains:

$$\tau ::= \text{bool} \mid \text{int} \mid \text{bytemap} \mid \& \tau \mid \&\text{mut } \tau$$

The main case is a reference to an integer cell:

$$r : \&\text{int} \quad q : \&\text{mut int}$$

Reference Triple

Symbolically, a reference carries:

$$r = \{\text{addr} : i, \text{value} : v, \text{promise} : p\}$$

- `addr`: location pointed to by the reference
- `value`: current value observed through the reference
- `promise`: value that will be committed on release

The `promise` field is unused for constant references, but keeping one shape makes the lowering uniform.

Record syntax is sugar for the lowering only — Tiny TAC itself has no tuples.

Borrow Commands

```
r := borrow M[i]
r, M2 := borrow_mut M[i]
q := borrow_ref r
q, r2 := borrow_ref_mut r
x := get_ref r
r2 := put_ref r, v
release r
```

Mutable borrowing returns both the reference and a continuation memory. Mutable reborrowing returns the child reference and the resumed parent reference.

Lowering: Direct Borrows

Constant borrow:

```
r := borrow M[i]
```

lowers to:

```
r := { addr: i,  
       value: M[i],  
       promise: havoc }
```

Mutable borrow:

```
r, M2 := borrow_mut M[i]
```

lowers to:

```
r := { addr: i,  
       value: M[i],  
       promise: havoc }  
M2 := M[i := r.promise]
```

The continuation memory is written **now**, with a havoc'd promise — a bet on the future final value.

Lowering: Use And Release

Read:

```
x := get_ref r
```

lowers to:

```
x := r.value
```

Write:

```
r2 := put_ref r, v
```

lowers to:

```
r2 := { addr: r.addr,  
        value: v,  
        promise: r.promise }
```

Release:

```
assume r.value == r.promise
```

release is where the bet is settled: the value observed at the end **is** the promised value.

The Puzzle, Lowered

```
entry:
  M := havoc
  i := havoc
  r := { addr: i, value: M[i], promise: havoc }
  M2 := M[i := r.promise]
  r2 := { addr: r.addr, value: 7, promise: r.promise }
  assume r2.value == r2.promise
  x := M2[i]
  ok := x == 7
  assert ok
```

Only ordinary assignments, a map store, and an assume — existing VCGen applies unchanged.

Why It Proves

$$M2 = M[i := \text{promise}(r)]$$

$$\text{value}(r2) = 7$$

$$\text{promise}(r2) = \text{promise}(r)$$

$$\text{value}(r2) = \text{promise}(r2)$$

Why It Proves

$$\begin{aligned}M2 &= M[i := \text{promise}(r)] \\ \text{value}(r2) &= 7 \\ \text{promise}(r2) &= \text{promise}(r) \\ \text{value}(r2) &= \text{promise}(r2)\end{aligned}$$

Chaining the last three equalities:

$$\text{promise}(r) = 7$$

So $M2$ — written back at borrow time — stores 7 at address i , and $x == 7$ follows. The havoc'd promise was **forced** by the release assumption.

Demo: The Whole Pipeline

Desugaring is a ttac -> ttac pass; the VC core never sees a reference:

```
$ ttac desugar safe_borrow_mut.ttac | ttac vcgen - --solve
unsat
$ ttac desugar unsafe_borrow_mut.ttac | ttac vcgen - --solve
sat
```

Demo: The Whole Pipeline

Desugaring is a ttac -> ttac pass; the VC core never sees a reference:

```
$ ttac desugar safe_borrow_mut.ttac | ttac vcgen - --solve
unsat
$ ttac desugar unsafe_borrow_mut.ttac | ttac vcgen - --solve
sat
```

One more thing the semantics tells us — forward execution **cannot** run a borrow program:

```
$ ttac run safe_borrow_mut.ttac
status: stopped (assume failed in block entry)
```

The interpreter cannot guess the promise. Only the solver — choosing all values at once — can. That is what makes it a **prophecy**.

Mutable Reborrow

```
r, M2 := borrow_mut M[i]
q, r2 := borrow_ref_mut r
q2 := put_ref q, 7
release q2
r3 := put_ref r2, 8
release r3
```

`q` borrows through `r`. After `q` is released, `r2` resumes the parent reference and can overwrite the final promised value.

Reborrow Lowering

```
q := { addr: r.addr, value: r.value, promise: havoc }  
r2 := { addr: r.addr, value: q.promise, promise: r.promise }
```

When `q` is released, it updates `r2.value`. When `r2` is released, it updates the original memory promise.

The same trick, nested: the parent's resumed view starts at the child's **promise**.

Reborrow Facts

$$\begin{aligned}M2 &= M[i := \text{promise}(r)] \\ \text{value}(q2) &= 7 \\ \text{value}(q2) &= \text{promise}(q) \\ \text{value}(r3) &= 8 \\ \text{promise}(r3) &= \text{promise}(r2) \\ \text{promise}(r2) &= \text{promise}(r)\end{aligned}$$

The final release implies $\text{promise}(r) = 8$, so $M2[i] == 8$ — the write through the child is visible to the parent, and the parent's final write wins.

Prophecy Variables

The `promise` field is a **prophecy variable**:

- chosen nondeterministically at borrow time
- constrained later at release time
- already written into the continuation memory

Lineage:

- Abadi and Lamport, *The Existence of Refinement Mappings* (1991) — prophecy variables for refinement proofs.
- Matsushita, Tsukada, Kobayashi, *RustHorn: CHC-based Verification for Rust Programs* (2020) — mutable borrows as (current, final) value pairs, no memory model.
- Priya and Gurfinkel, *Ownership in low-level intermediate representation* (FMCAD 2024) — the idea in a BMC setting, mixed with an address-map memory model. Tiny TAC follows this pattern.

VCGen Boundary

After lowering, VCGen only sees ordinary Tiny TAC:

- assignments
- map reads and stores
- assumptions
- assertions
- havoc values

Borrow **well-formedness** (liveness, exclusivity, reborrow lifetimes) is a separate side condition — checked before VCGen, or encoded as extra obligations. Candidate disciplines: Stacked Borrows, Tree Borrows.

Open Extensions

- **Finite-region borrows** — borrow a byte range, not a single cell: base + size + per-word value/promise. Question: keeping the encoding compact.
- **Reference shadow memory** — store references in memory via ghostmap shadows for addr/value/promise/permission. Question: which metadata must be shadowed, and how shadows stay synchronized.
- **Reference semantics and well-formedness** — pick a borrow model (Stacked/Tree Borrows), state the rules, decide: reject before VCGen or encode as obligations.

Each is small enough to state independently, but substantial enough to be a useful project.